



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Clase 10: Clean Code (Cap. 10) [SOLID parte 3]

Francisco Gazitúa Requena (fco_gazitua_requena@uc.cl)

IIC2113 Diseño Detallado de Software

08 de Septiembre, 2026

Classes

with Jeff Langr



“So far in this book we have focused on how to write lines and blocks of code well. We have delved into proper composition of functions and how they interrelate. But for all the attention to the expressiveness of code statements and the functions they comprise, we still don’t have clean code until we’ve paid attention to higher levels of code organization. Let’s talk about clean classes”

– Robert C. Martin & Jeff Langr.

Este capítulo nos presenta con uno de los mandamientos más importantes de Clean Code:

Este capítulo nos presenta con uno de los mandamientos más importantes de Clean Code:

THE FIRST RULE OF CLASSES IS THAT THEY SHOULD BE SMALL.

Este capítulo nos presenta con uno de los mandamientos más importantes de Clean Code:

**THE FIRST RULE OF CLASSES IS THAT THEY SHOULD BE SMALL.
THE SECOND RULE OF CLASSES IS THAT THEY SHOULD BE SMALLER
THAN THAT.**

Este capítulo nos presenta con uno de los mandamientos más importantes de Clean Code:

**THE FIRST RULE OF CLASSES IS THAT THEY SHOULD BE SMALL.
THE SECOND RULE OF CLASSES IS THAT THEY SHOULD BE SMALLER
THAN THAT.**

Al igual que con las funciones, queremos que nuestras clases hagan solo una cosa

Este capítulo nos presenta con uno de los mandamientos más importantes de Clean Code:

**THE FIRST RULE OF CLASSES IS THAT THEY SHOULD BE SMALL.
THE SECOND RULE OF CLASSES IS THAT THEY SHOULD BE SMALLER
THAN THAT.**

Al igual que con las funciones, queremos que nuestras clases hagan solo una cosa
¿Por qué es recomendable que una clase sea pequeña?

Este capítulo nos presenta con uno de los mandamientos más importantes de Clean Code:

**THE FIRST RULE OF CLASSES IS THAT THEY SHOULD BE SMALL.
THE SECOND RULE OF CLASSES IS THAT THEY SHOULD BE SMALLER
THAN THAT.**

Al igual que con las funciones, queremos que nuestras clases hagan solo una cosa
¿Por qué es recomendable que una clase sea pequeña?

- Entender una cosa es más fácil que entender dos cosas.
- Es más fácil reutilizar clases pequeñas.
- Es menos trabajo leer clases pequeñas

Al igual que con funciones, la clave es que tus clases sean pequeñas.

```
2 public class SuperDashboard{  
3     public Component getLastFocusedComponent() { /* ... */}  
4     public void setLastFocused(Component lastFocused) { /* ... */}  
5     public int getMajorVersionNumber() { /* ... */}  
6     public int getMinorVersionNumber() { /* ... */}  
7     public int getBuildNumber() { /* ... */}  
8 }
```

Al igual que con funciones, la clave es que tus clases sean pequeñas.

```
2 public class SuperDashboard{  
3     public Component getLastFocusedComponent() { /* ... */}  
4     public void setLastFocused(Component lastFocused) { /* ... */}  
5     public int getMajorVersionNumber() { /* ... */}  
6     public int getMinorVersionNumber() { /* ... */}  
7     public int getBuildNumber() { /* ... */}  
8 }
```

¿Es esta clase suficientemente pequeña?

Al igual que con funciones, la clave es que tus clases sean pequeñas.

```
2 public class SuperDashboard{
3     public Component getLastFocusedComponent() { /* ... */}
4     public void setLastFocused(Component lastFocused) { /* ... */}
5     public int getMajorVersionNumber() { /* ... */}
6     public int getMinorVersionNumber() { /* ... */}
7     public int getBuildNumber() { /* ... */}
8 }
```

¿Es esta clase suficientemente pequeña?

Cada clase debiera tener una sola *responsabilidad* pero SuperDashboard tiene dos:

- 1 Proveer acceso al último componente con que interactuó el usuario.
- 2 Llevar un registro con la versión actual del programa.

Para medir el largo de una clase **NO** contamos líneas de código ni métodos públicos. Contamos el número de responsabilidades que tiene.

Para medir el largo de una clase **NO** contamos líneas de código ni métodos públicos. Contamos el número de responsabilidades que tiene.

The Single Responsibility Principle (SRP)

The Single Responsibility Principle (SRP) states that a class or module should have one, and only one, reason to change.

Nuestras clases deben tener un solo motivo para cambiar.

The Single Responsibility Principle (SRP)

The Single Responsibility Principle (SRP)

"A class should have just one reason to change."

The Single Responsibility Principle (SRP)

The Single Responsibility Principle (SRP)

"A class should have just one reason to change."

Existe otra formulación (quizas más completa):

The Single Responsibility Principle (SRP)

"A module should be responsible to one, and only one, actor."

The Single Responsibility Principle (SRP)

The Single Responsibility Principle (SRP)

"A class should have just one reason to change."

Existe otra formulación (quizas más completa):

The Single Responsibility Principle (SRP)

"A module should be responsible to one, and only one, actor."

Este principio busca solucionar dos problemas:

The Single Responsibility Principle (SRP)

The Single Responsibility Principle (SRP)

"A class should have just one reason to change."

Existe otra formulación (quizas más completa):

The Single Responsibility Principle (SRP)

"A module should be responsible to one, and only one, actor."

Este principio busca solucionar dos problemas:

- 1 Las clases se vuelven tan grandes que tienes que navegar por un mar de código irrelevante hasta encontrar la funcionalidad que necesitas

The Single Responsibility Principle (SRP)

The Single Responsibility Principle (SRP)

"A class should have just one reason to change."

Existe otra formulación (quizas más completa):

The Single Responsibility Principle (SRP)

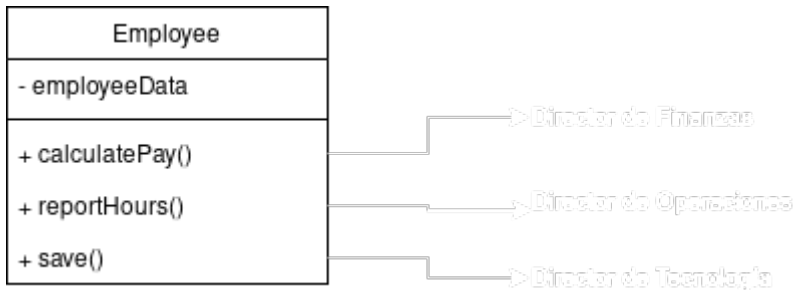
"A module should be responsible to one, and only one, actor."

Este principio busca solucionar dos problemas:

- 1 Las clases se vuelven tan grandes que tienes que navegar por un mar de código irrelevante hasta encontrar la funcionalidad que necesitas
- 2 Si una clase hace más de una cosa, cuando quieras cambiar solo una de estas corres el riesgo de romper partes no relacionadas

Ejemplo

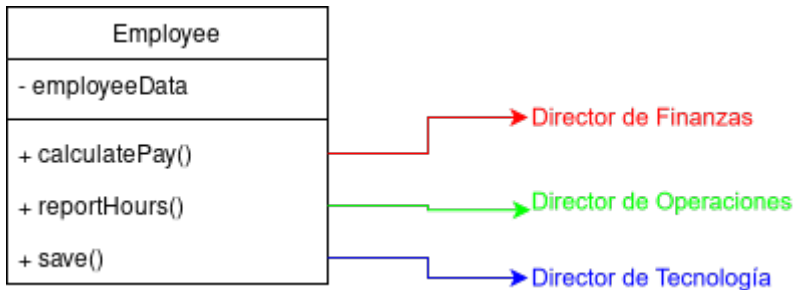
Imagina que trabajamos en software para una empresa que cuenta con la siguiente clase:



¿Qué actores dentro de la empresa le importa cada método?

Ejemplo

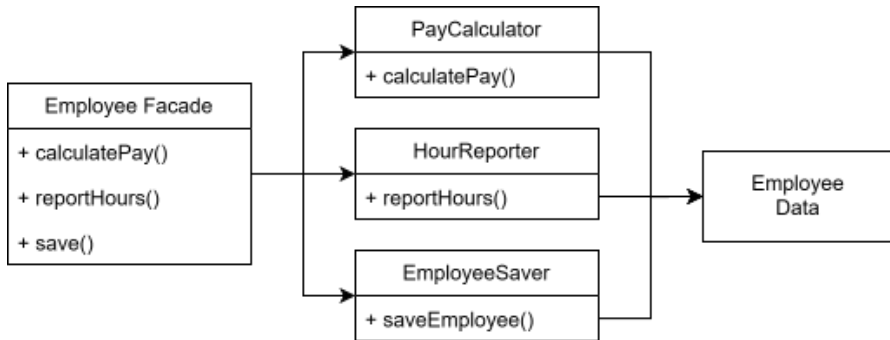
Imagina que trabajamos en software para una empresa que cuenta con la siguiente clase:



Esta clase tiene 3 razones para cambiar

Ejemplo

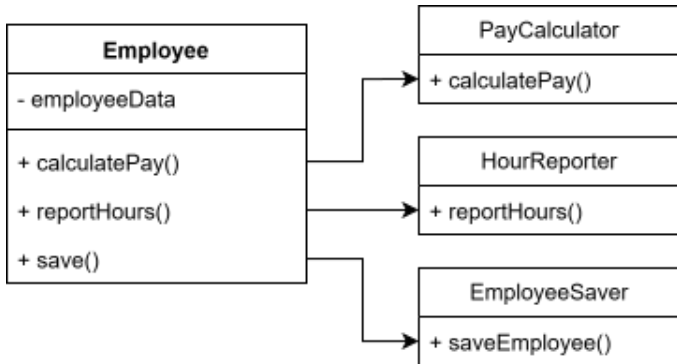
Una posible solución, separar en más clases:



EmployeeFacade delega trabajo a las otras clases

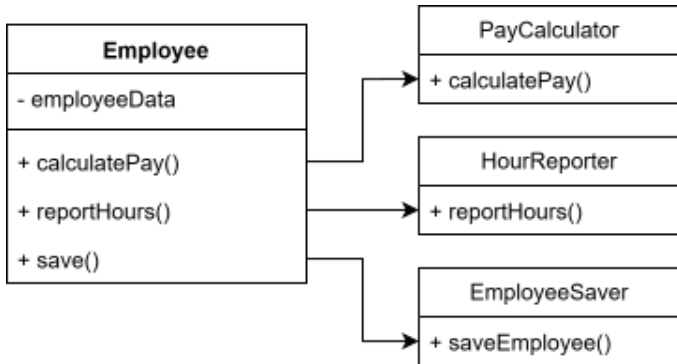
Ejemplo

Otra solución similar, las reglas de negocio están cerca de la Data:



Ejemplo

Otra solución similar, las reglas de negocio están cerca de la Data:



Ambas soluciones se basan en separar responsabilidades en múltiples clases

Volvamos al primer ejemplo:

```
2 public class SuperDashboard{
3     public Component getLastFocusedComponent() { /* ... */}
4     public void setLastFocused(Component lastFocused) { /* ... */}
5     public int getMajorVersionNumber() { /* ... */}
6     public int getMinorVersionNumber() { /* ... */}
7     public int getBuildNumber() { /* ... */}
8 }
```

¿Qué motivos hay para cambiar esta clase?

Volvamos al primer ejemplo:

```
2 public class SuperDashboard{
3     public Component getLastFocusedComponent() { /* ... */}
4     public void setLastFocused(Component lastFocused) { /* ... */}
5     public int getMajorVersionNumber() { /* ... */}
6     public int getMinorVersionNumber() { /* ... */}
7     public int getBuildNumber() { /* ... */}
8 }
```

¿Qué motivos hay para cambiar esta clase?

- 1 Proveer acceso al último componente con que interactuó el usuario.
- 2 Llevar un registro con la versión actual del programa.

Volvamos al primer ejemplo:

```
2 public class SuperDashboard{
3     public Component getLastFocusedComponent() { /* ... */}
4     public void setLastFocused(Component lastFocused) { /* ... */}
5     public int getMajorVersionNumber() { /* ... */}
6     public int getMinorVersionNumber() { /* ... */}
7     public int getBuildNumber() { /* ... */}
8 }
```

¿Qué motivos hay para cambiar esta clase?

Solución:

```
2 public class Version {
3     public int getMajorVersionNumber() { /* ... */}
4     public int getMinorVersionNumber() { /* ... */}
5     public int getBuildNumber() { /* ... */}
6 }
```

Mantener clases con alta cohesión resulta en clases pequeñas. Una clase es cohesiva cuando:

- Tiene pocos atributos.
- Sus métodos usan todos (o casi todos) los atributos de la clase.

Mantener clases con alta cohesión resulta en clases pequeñas. Una clase es cohesiva cuando:

- Tiene pocos atributos.
- Sus métodos usan todos (o casi todos) los atributos de la clase.

Intuición:

Si todos los métodos usan todos (o casi todos) los atributos de la clase entonces tu clase es una unidad lógica completa, con una sola responsabilidad.

```
2 public class Stack {
3     private int topOfStack = 0;
4     List<int> elements = new List<int>();
5
6     public int size()
7         => topOfStack;
8
9     public void push(int element) {
10         topOfStack++;
11         elements.add(element);
12     }
13
14     public int pop() {
15         if (topOfStack == 0)
16             throw new PoppedWhenEmpty();
17         int element = elements.get(--topOfStack);
18         elements.remove(topOfStack);
19         return element;
20     }
21 }
```

Veamos otro ejemplo. ¿Es cohesiva?:

```
2 public class CustomerOrder(int orderId, Customer customer, double
   orderIdAmount)
3 {
4     private int _orderId = orderId;
5     private Customer _customer = customer;
6     private double _orderIdAmount = orderIdAmount;
7
8     public bool IsOrderValid()
9         => _orderId > 0;
10
11     public double CalculateTax()
12         => _orderIdAmount * 0.07;
13
14     public void AssignLoyaltyPoints()
15         => _customer.LoyaltyPoints += (int)(_customer.loyaltyModifier
16         * 10);
17 }
```

Intentemos hacerla más cohesiva:

```
2 public class CustomerOrder(OrderInfo orderInfo)
3 {
4     private OrderInfo = orderInfo;
5
6     public bool IsOrderValid()
7         => orderInfo.orderId > 0;
8
9     public double CalculateTax()
10         => orderInfo.orderAmount * 0.07;
11
12     public void AssignLoyaltyPoints()
13         => orderInfo.Customer.LoyaltyPoints += (int)(orderInfo.
14         Customer.loyaltyModifier * 10);
15 }
```

Intentemos hacerla más cohesiva:

```
2 public class CustomerOrder(OrderInfo orderInfo)
3 {
4     private OrderInfo = orderInfo;
5
6     public bool IsOrderValid()
7         => orderInfo.orderId > 0;
8
9     public double CalculateTax()
10         => orderInfo.orderAmount * 0.07;
11
12     public void AssignLoyaltyPoints()
13         => orderInfo.Customer.LoyaltyPoints += (int)(orderInfo.
14         Customer.loyaltyModifier * 10);
15 }
```

¿Es cohesiva?

Intentemos hacerla más cohesiva:

```
2 public class CustomerOrder(OrderInfo orderInfo)
3 {
4     private OrderInfo = orderInfo;
5
6     public bool IsOrderValid()
7         => orderInfo.orderId > 0;
8
9     public double CalculateTax()
10         => orderInfo.orderAmount * 0.07;
11
12     public void AssignLoyaltyPoints()
13         => orderInfo.Customer.LoyaltyPoints += (int)(orderInfo.
14         Customer.loyaltyModifier * 10);
15 }
```

¿Es cohesiva? ¿Tiene una sola razón de cambio?

Lo más interesante del capítulo es que nos enseña un procedimiento para limpiar nuestro código que resulta en varias clases pequeñas con una responsabilidad.

Lo más interesante del capítulo es que nos enseña un procedimiento para limpiar nuestro código que resulta en varias clases pequeñas con una responsabilidad.

Algoritmo para limpiar tu código:

- 1 Parte de una función que sea demasiado grande.
- 2 Extrae funciones chicas de esa función.
- 3 Esas funciones chicas tendrán muchos parámetros.
- 4 Reduce el número de parámetros transformándolos en atributos.
- 5 Ahora tienes una gran clase con baja cohesión (con muchos atributos y métodos que solo usan subconjuntos de esos atributos).
- 6 Si un conjunto de métodos solo usa un subconjunto de atributos, extráelos y ponlos en una nueva clase. Esto aumentará la cohesión de tu sistema.
- 7 Repetir hasta que no tenga sentido seguir extrayendo métodos/clases.

¿Cuáles son los descuentos típicos en el proyecto debido al cap. 10?

Descuentos más comunes:

- Clases con más de una responsabilidad.

Code smells para detectar clases con varias responsabilidades:

- Clases muy largas.
- Clases poco cohesivas (muchos atributos que son usados por pocos métodos).

Ejemplo del proyecto

Ejemplo

Digamos que diseñamos los efectos del juego de la siguiente manera:

```
1 public class PhysDamageEffect : IEffect
2 {
3     private int _appliedDamage;
4     public void Apply(GameState state)
5     {
6         /* Damage Logic */
7         string[] logMessages = CreateLogMessages(state);
8         foreach (string message in logMessages)
9             state.EffectLog.Add(message);
10    }
11    private string[] CreateLogMessages(GameState state)
12        => new string[]
13        {
14            $"{state.ActingUnit.Name} ataca a {state.Target.name}",
15            $"{state.Target.Name} es débil contra el ataque de {state.
16            ActingUnit.Name}",
17            $"{state.Target.Name} recibe {_appliedDamage} de daño",
18            $"{state.Target.Name} termina con HP:{state.Target.Stats.CurrentHp}
19           }/{state.Target.Stats.HP}"
16            };
17    }
```

Ejemplo

La clase GameState sería un modelo y se vería así:

Ejemplo

La clase GameState sería un modelo y se vería así:

```
1 public class GameState
2 {
3     public List<string> EffectLog { get; set; } = new List<string>();
4     public Unit ActingUnit { get; set; }
5     public Unit Target { get; set; }
6     /* Otras propiedades relevantes */
7 }
```

Ejemplo

La clase GameState sería un modelo y se vería así:

```
1 public class GameState
2 {
3     public List<string> EffectLog { get; set; } = new List<string>();
4     public Unit ActingUnit { get; set; }
5     public Unit Target { get; set; }
6     /* Otras propiedades relevantes */
7 }
```

Luego, en nuestra vista anunciaríamos el resultado del efecto de la siguiente manera:

Ejemplo

La clase GameState sería un modelo y se vería así:

```
1 public class GameState
2 {
3     public List<string> EffectLog { get; set; } = new List<string>();
4     public Unit ActingUnit { get; set; }
5     public Unit Target { get; set; }
6     /* Otras propiedades relevantes */
7 }
```

Luego, en nuestra vista anunciaríamos el resultado del efecto de la siguiente manera:

```
1 public class SMTConsoleView : IView
2 {
3     public void AnnounceResults(GameState state)
4     {
5         foreach (string message in state.EffectLog)
6             Console.WriteLine(message);
7     }
8     /* Otros métodos */
9 }
```

Ejemplo

La clase GameState sería un modelo y se vería así:

```
1 public class GameState
2 {
3     public List<string> EffectLog { get; set; } = new List<string>();
4     public Unit ActingUnit { get; set; }
5     public Unit Target { get; set; }
6     /* Otras propiedades relevantes */
7 }
```

Luego, en nuestra vista anunciaríamos el resultado del efecto de la siguiente manera:

```
1 public class SMTConsoleView : IView
2 {
3     public void AnnounceResults(GameState state)
4     {
5         foreach (string message in state.EffectLog)
6             Console.WriteLine(message);
7     }
8     /* Otros métodos */
9 }
```

Esto rompe el SRP ¿Cómo?

Actividad

(Opcional)

Objetivo

Limpia el código Automata

Objetivo

Limpia el código Automata

Contexto:

Los autómatas son grafos que aceptan o rechazan palabras. Un autómata está compuesto por estados y transiciones. El estado inicial es q_0 y las transiciones $\delta(q, \sigma) = q'$ indica el siguiente estado q' dado que el estado actual es q y el caracter es σ . El automata procesa la palabra caracter a caracter cambiando su estado. Si al terminar la palabra el automata se encuentra en un estado de aceptación, la palabra es aceptada. La palabra es rechazada en otro caso.

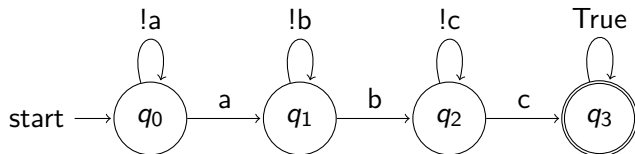
Actividad

Objetivo

Limpia el código Automata

Contexto:

Los autómatas son grafos que aceptan o rechazan palabras. Un autómata está compuesto por estados y transiciones. El estado inicial es q_0 y las transiciones $\delta(q, \sigma) = q'$ indica el siguiente estado q' dado que el estado actual es q y el caracter es σ . El automata procesa la palabra caracter a caracter cambiando su estado. Si al terminar la palabra el automata se encuentra en un estado de aceptación, la palabra es aceptada. La palabra es rechazada en otro caso.



abcb → Aceptada!

cambalache → Aceptada!

cba → Rechazada!

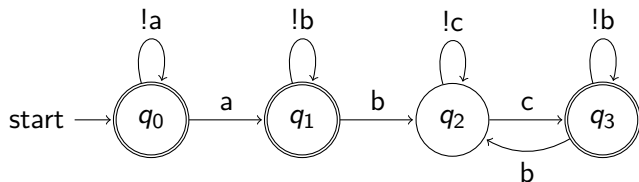
intercambio → Rechazada!

Objetivo

Limpia el código Automata

Contexto:

Los autómatas son grafos que aceptan o rechazan palabras. Un autómata está compuesto por estados y transiciones. El estado inicial es q_0 y las transiciones $\delta(q, \sigma) = q'$ indica el siguiente estado q' dado que el estado actual es q y el caracter es σ . El automata procesa la palabra caracter a caracter cambiando su estado. Si al terminar la palabra el automata se encuentra en un estado de aceptación, la palabra es aceptada. La palabra es rechazada en otro caso.



abcb → Rechazada!

cambalache → Aceptada!

cba → Aceptada!

intercambio → Rechazada!

El código lee autómatas en el siguiente formato: DFA/02.txt

```
2 0,1,3
3 0,a,1
4 1,b,2
5 2,c,3
6 3,b,2
```

La primera línea indica los estados de aceptación (0, 1 y 3). Las siguientes líneas indican las transiciones. En el ejemplo, la primera transición es $\delta(q_0, a) = q_1$. La segunda es $\delta(q_1, b) = q_2$. Etc. Las transiciones no definidas son self-loops. Por ejemplo, $\delta(q_0, b) = q_0$.

Actividad

El código hace lo siguiente:

```
2 public class Automaton {
3     public bool IsWordValid(string fileDFA, string word) {
4         StreamReader reader = new StreamReader(fileDFA);
5
6         /* ... Lee los estados de aceptación ... */
7
8         /* ... Lee las transiciones ... */
9
10        int state = 0;
11        foreach (var symbol in word)
12            if(transitions.ContainsKey((state, symbol)))
13                state = transitions[(state, symbol)];
14
15        return acceptingStates.Contains(state);
16    }
17 }
```

Este código lee los estados de aceptación:

```
2 HashSet<int> acceptingStates = new HashSet<int>();
3 string firstLine = reader.ReadLine();
4 string[] acceptingStateStrs = firstLine.Split(',');
5 foreach (var acceptingStateStr in acceptingStateStrs) {
6     int acceptingState = Convert.ToInt32(acceptingStateStr);
7     acceptingStates.Add(Convert.ToInt32(acceptingState));
8 }
```

Este código lee las transiciones:

```
10 Dictionary<(int, char), int> transitions = new ();
11 while (!reader.EndOfStream){
12     string transitionStr = reader.ReadLine();
13     string[] transitionComponents = transitionStr.Split(',');
14     int startingState = Convert.ToInt32(transitionComponents[0]);
15     char symbol = Convert.ToChar(transitionComponents[1]);
16     int endingState = Convert.ToInt32(transitionComponents[2]);
17     transitions[(startingState, symbol)] = endingState;
18 }
```

Actividad

Finalmente, el código cuenta con tests (que podría ser necesario modificar):

```
2 [Theory]
3 [InlineData("01.txt", "abc")]
4 [InlineData("01.txt", "sdwaagfbfscds")]
5 [InlineData("02.txt", "")]
6 [InlineData("02.txt", "aaa")]
7 public void IsWordValid_ThisWordShouldBeValid(string fileDFA, string
    word){
8     fileDFA = GetPathToDFA(fileDFA);
9     Automaton dfa = new Automaton();
10
11     bool isWordValid = dfa.IsWordValid(fileDFA, word);
12
13     Assert.True(isWordValid);
14 }
```

Actividad

Finalmente, el código cuenta con tests (que podría ser necesario modificar):

```
2 [Theory]
3 [InlineData("01.txt", "abc")]
4 [InlineData("01.txt", "sdwaagfbfscds")]
5 [InlineData("02.txt", "")]
6 [InlineData("02.txt", "aaa")]
7 public void IsWordValid_ThisWordShouldBeValid(string fileDFA, string
    word){
8     fileDFA = GetPathToDFA(fileDFA);
9     Automaton dfa = new Automaton();
10
11     bool isWordValid = dfa.IsWordValid(fileDFA, word);
12
13     Assert.True(isWordValid);
14 }
```

Así que puedes limpiar el código con confianza